Football Player Detection in Video Sequences

From Classical Computer Vision to Modern Deep Learning Detectors

1. Abstract

Player detection represents the foundational step of any football video analysis pipeline, enabling downstream tasks such as tracking, feature extraction, tactical breakdown, and event recognition. In this work, we investigate and compare two approaches for football player detection in broadcast video sequences: a classical computer vision pipeline based on background subtraction and morphological filtering, and a deep learning–based method using a fine-tuned YOLO detector. To ensure a fair comparison between the two approaches, we introduce a unified evaluation framework that ensures consistent and comparable precision-recall metrics across both methods. Experimental results on real match sequences show that the fine-tuned YOLO model achieves higher precision and recall than the classical pipeline while maintaining real-time performance. The proposed methodology is general and can be extended to player detection in other sports or integrated into more advanced multi-player tracking and analysis systems.

2. Introduction

In recent years, computer vision has become an essential tool for sports analytics, enabling applications ranging from performance monitoring to tactical evaluation and opponent analysis. Traditional approaches often rely on wearable sensors placed on athletes to collect motion data, which can then be processed to extract performance indicators. However, sensor-based systems introduce additional costs, may interfere with athletes' movements, and cannot be applied to opponents during competitive matches. These limitations have motivated the development of vision-based solutions capable of extracting meaningful information directly from video footage. A fundamental component of any vision-based sports analysis pipeline is player detection, which serves as the basis for subsequent tasks such as tracking, feature extraction, tactical breakdown, and event recognition. Accurate detection is particularly challenging in football due to frequent occlusions, rapid motion, varying camera viewpoints, and heterogeneous visual conditions typical of broadcast footage. To address these challenges, two main families of approaches have emerged: classical computer vision methods and modern deep learning–based detectors. In the following sections, we describe in detail the two detection pipelines, outlining their underlying principles, implementation choices, and practical limitations. We then present the unified evaluation framework and discuss the experimental setup used to assess the performance of both approaches on real match footage.

3. Related Work

Early approaches to player detection in sports video can be broadly categorized into appearance-based and motion-based methods. Appearance-based techniques typically rely on color cues, texture descriptors, or handcrafted visual features to distinguish players from the field. Motion-based approaches, on the other hand, exploit temporal information through background subtraction or frame differencing to isolate moving objects. While computationally lightweight, these classical methods often struggle in challenging broadcast scenarios involving occlusions, cluttered backgrounds, or players in close proximity, and their performance is sensitive to illumination changes and camera motion. With the rise of deep learning, a wide range of neural-network-based detectors has been applied to sports analytics. These include both two-stage and one-stage architectures, as well as models that integrate temporal information or multi-scale feature extraction. Deep learning–based approaches generally offer greater robustness to scale variation, viewpoint changes, and heterogeneous visual conditions typical of football broadcasts. They also tend to handle difficult situations, such as overlapping players, players on the ground, or crowded penalty-area scenes, more effectively than classical methods. Domain-specific fine-tuning and data-driven feature learning further contribute to improved detection performance across different sports.

4. Methods

4.1 Dataset

We base our analysis on two video sequences selected to represent notably different visual conditions. The first sequence was extracted from a public dataset released as part of a FIGC–CNR acquisition campaign on soccer video analysis, commonly referred to in the literature as the ISSIA-CNR Soccer dataset. The footage is recorded under controlled and relatively ideal conditions: the camera is static, the field is clearly visible, and the scene contains few distracting elements around the pitch. Players appear relatively large in the frame, and only limited occlusions occur. From the available material, we selected a clip of approximately twenty seconds that provides stable, high-quality imagery suitable for evaluating detection performance in favourable settings. The second sequence was taken from a full-match tactical-camera recording of the Como–Roma game available on YouTube. This sequence reflects far more challenging and realistic broadcast conditions: the camera covers the entire pitch, players appear smaller and more distant, and frequent occlusions occur. The scene includes spectators, staff members, and other elements around the field, as well as complex backgrounds such as stands and advertising boards. In addition, the camera performs panning movements, and the lighting and contrast vary throughout the clip. A

twenty-second segment was extracted to capture these difficulties and assess the robustness of the detection methods. Together, the two sequences provide a complementary evaluation setup: the first clip represents an idealized scenario, while the second one captures the complexity of real-world football footage.

4.2 Classical Computer Vision Approach

As a classical baseline, we employ MOG2, a Gaussian Mixture-based background-subtraction model widely used for motion segmentation. MOG2 models each pixel as a mixture of Gaussians and updates the background online, making it suitable for real-time applications and for scenes with moderate illumination changes. Starting from this common motion mask, we evaluate three variants of the pipeline obtained by combining color-based filtering, spatial constraints, and morphological operations. Several components of the pipeline are hard-coded, including the HSV range used to identify the green pitch, the minimum bounding-box area, computed adaptively from the frame size, and the morphological kernels applied to clean the masks. Different configurations are used for the fixed-camera sequence and the moving-camera one, reflecting their different visual characteristics.

Not green case. The first variant intersects the MOG2 motion mask with a color-based filter that removes pixels corresponding to the green field. The resulting mask is refined using morphological opening and closing and player candidates are extracted by contour detection. Contours are then filtered using adaptive thresholds on area, height and aspect-ratio before being converted into bounding boxes.

Region of interest case. The second variant uses the green pitch only to identify the playing field and restrict motion analysis to that region. A field mask is obtained in HSV, and the largest connected component is selected and expanded to form a clean ROI. Motion is then detected with MOG2, and the final mask is obtained by intersecting the motion mask with the ROI. Bounding boxes are extracted and filtered using the same adaptive criteria as in the first variant.

Double filter case. The third variant combines the two previous strategies. A field ROI is first extracted from the HSV segmentation, and a non-green mask is computed to suppress field pixels. The final player mask is obtained by intersecting the motion mask, the non-green mask, and the ROI. Bounding boxes are then extracted and filtered using the same constraints as in the other variants.

The figure shows one representative frame per sequence: the ideal scenario on the left and the challenging scenario on the right. For each case, we display the same frame processed with the three variants of our pipeline: *not-green mask*, *region of interest*, and *double filter*.

4.3 Pretrained YOLO Approach

YOLO - You Only Look Once - is a family of real-time object detectors based on a single-stage convolutional architecture that jointly predicts bounding boxes and class probabilities. Its efficiency and accuracy make it a standard baseline for many detection tasks, including sports analytics. However, pretrained YOLO models are typically trained on generic datasets such as COCO, which contain few or no soccer-specific samples, and therefore may struggle with small players, crowded scenes, and long-shot tactical views. Among the available YOLOv8 variants, we selected the YOLOv8-medium model as a trade-off between accuracy and computational cost. Preliminary tests showed that lighter models - n, s - produced unstable detections in long-shot frames, while heavier models - l, x - offered limited gains relative to their increased inference time. YOLOv8m provided the most balanced performance across both sequences.

YOLO only case. We first evaluate the pretrained YOLOv8-medium model using full-frame inference. Each frame is processed at a resolution of 1280 pixels on the longest side, with a confidence threshold of 0.35 and a non-maximum suppression IoU threshold of 0.5. Only detections belonging to the *person* class are retained. No spatial or color-based filtering is applied, and the model is used exactly as provided, without any fine-tuning. The resulting bounding boxes are stored frame-by-frame and used directly for evaluation.

YOLO and region of interest case. We also evaluate a variant in which YOLO detections are filtered using a field-based region of interest. The pitch is segmented in HSV using a fixed green range, and the largest connected component is extracted and converted into a convex hull. YOLOv8m is then applied to the full frame, and only detections whose bounding-box center lies inside the field hull are retained. This simple spatial constraint removes false positives occurring outside the green area while leaving the detector unchanged. A slightly higher confidence threshold of 0.40 is used in this variant. Since the ROI constraint already removes detections outside the green field, the threshold can be increased without reducing recall. This helps suppress duplicate boxes on true players, an issue that becomes more frequent at lower thresholds.

The figure shows the same representative frames obtained with the two variants of our second approach: plain *YOLO* and the combination of *YOLO with the region of interest*.

4.4 Fine-tuned YOLO Approach

While pretrained YOLO models provide a strong baseline, they are not specifically optimized for long-shot soccer footage. In our sequences, players appear small and

often blend with the background, and the pretrained detector - trained on the generic COCO dataset - does not always adapt perfectly to this visual domain. In particular, when applied without spatial constraints, the model may produce some false positives outside the playing area. For these reasons, we fine-tuned YOLOv8m on a custom dataset derived directly from our two sequences, with the goal of adapting the detector to the specific appearance and scale of players in our scenario.

Model Training

To prepare the training data, we sampled one frame every two from both sequences, obtaining 274 annotated frames for the moving-camera video and 246 for the fixed-camera one. To avoid temporal redundancy and prevent the model from overfitting to short-term motion patterns, each sequence was divided into non-overlapping temporal blocks. The fixed-camera sequence was partitioned into 18 blocks, while the moving-camera sequence was divided into 16 blocks. For the moving-camera sequence, we adopted a 12–2–2 split (training–test–validation), while for the fixed-camera one, we used a 11–2–1 split. In both cases, blocks assigned to different partitions were separated by two “gap” blocks, ensuring that temporally adjacent frames never fall into different splits. This strategy prevents temporal leakage and enforces a clear separation between training and evaluation data. After splitting each sequence independently, the training blocks from both videos were merged into a single training set, and the same procedure was applied to the validation and test sets, allowing the model to learn from both viewpoints while preserving strict temporal isolation. The model was trained using the standard YOLOv8 pipeline. We initialized from the pretrained YOLOv8m checkpoint and optimized the network for 50 epochs using the AdamW optimizer and a cosine learning-rate schedule. Training was performed at a resolution of 1280 pixels with a batch size of 2, using rectangular training to preserve the aspect ratio of the original frames. Standard YOLOv8 augmentations were enabled. All experiments were run on a single GPU with two dataloader workers, and early stopping was automatically applied based on validation performance.

Model Evaluation

We evaluate all detection methods using a unified frame-level matching procedure based on Intersection over Union (IoU). For each predicted bounding box, we compute its IoU with all ground-truth boxes in the frame and select the ground-truth box with the highest overlap. A prediction is counted as a true positive (TP) if this maximum IoU is at least 0.5 and the corresponding ground-truth box has not been matched previously. If the matched ground-truth box is already assigned to another prediction, the prediction is counted as a false positive (FP). Predictions whose best IoU is below 0.5 are also counted as false positives. Ground-truth boxes that are not matched by any prediction

are counted as false negatives (FN). Although inference is run on the entire sequence, evaluation is performed exclusively on the frames belonging to the test partition. Each sequence contributes 30 test frames, and TP, FP, and FN are accumulated independently within each sequence, ensuring that the metrics reflect performance under both viewpoints without mixing the two test sets. Moreover, ignoring training frames prevents temporal leakage and guarantees that the reported metrics reflect true generalization performance. Since SP methods do not output confidence scores, all predictions are evaluated at a single operating point. We sum TP, FP, and FN across all test frames of each sequence and compute global precision and recall as:

$$Precision = TP / (TP+FP) \qquad Recall = TP / (TP+FN)$$

Average Precision (AP) reduces to the single-point value:

$$AP = Precision \times Recall.$$

YOLO predictions include confidence scores, enabling the computation of a full precision–recall curve. All predictions from all test frames are sorted by confidence, and cumulative TP and FP counts are used to compute precision and recall at each confidence level. The AP is then obtained as the area under the precision–recall curve:

$$AP = \int Precision(Recall)\, dRecall$$

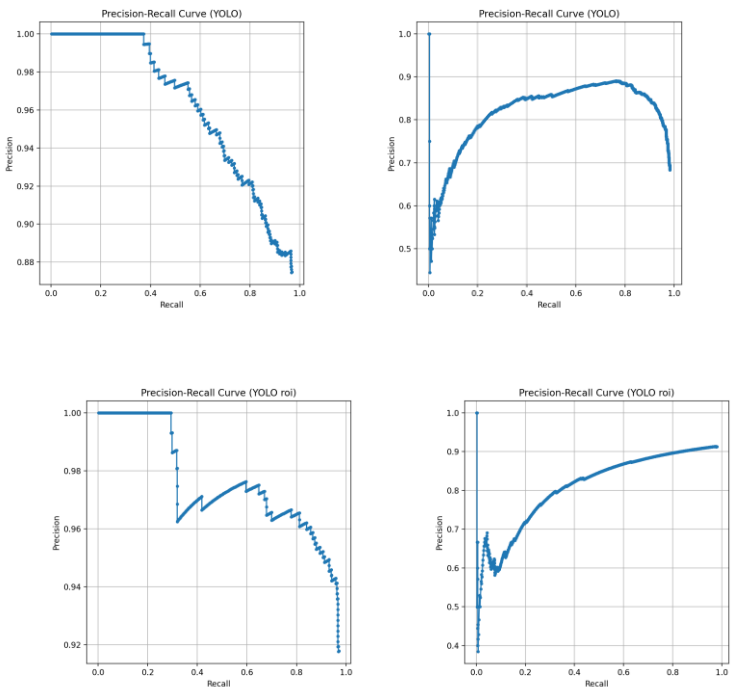
This yields a standard AP@0.5 metric consistent with common object-detection benchmarks. To ensure a fair comparison across all methods, the fine-tuned YOLO model is evaluated using the same IoU-based matching procedure described above, rather than relying on YOLO's internal evaluation module. This guarantees all methods are assessed under identical conditions; in fact, although YOLO's built-in evaluator yields similar results, small differences arise from internal preprocessing and AP interpolation.

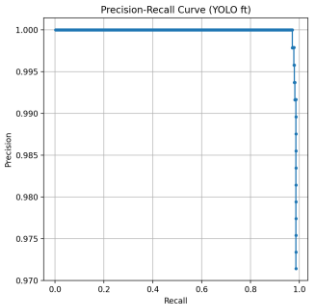
The figure shows the same representative frames obtained with the *fine-tuned YOLO model*.

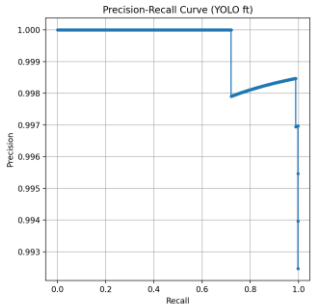
5. Results

Method	Precision (B)	Recall (B)	AP (B)	Precision (A)	Recall (A)	AP (A)
not green	0.830	0.867	0.720	0.128	0.576	0.074
roi	0.902	0.801	0.723	0.441	0.717	0.316
double	0.957	0.867	0.830	0.613	0.785	0.481
yolo	0.875	0.969	0.932	0.683	0.982	0.802
yolo - roi	0.918	0.971	0.946	0.913	0.980	0.793
yolo ft	0.971	0.986	0.983	0.992	0.997	0.995

Across the two evaluation sequences, the classical pipelines exhibit strong dependence on viewpoint: they perform reasonably well in the fixed sequence (B), but their performance drops sharply in moving sequence (A), confirming their limited robustness to perspective changes. In contrast, the pretrained YOLO model already generalizes substantially better, achieving high recall and competitive AP in both sequences without any adaptation. Introducing a ROI improves cumulative precision by removing distant false positives but does not increase AP in the challenging sequence A. This is because the ROI is defined as the green area rather than the actual playing field, and therefore still includes coaches, assistants, and the linesman. These individuals are detected with high confidence and are not annotated in the ground truth, generating high-confidence false positives that affect multiple points of the precision–recall curve. Finally, the fine-tuned YOLO model clearly dominates all other methods, achieving near-perfect AP in both sequences and demonstrating excellent robustness to viewpoint changes, effectively learning to distinguish relevant players from non-annotated individuals at the field boundary.

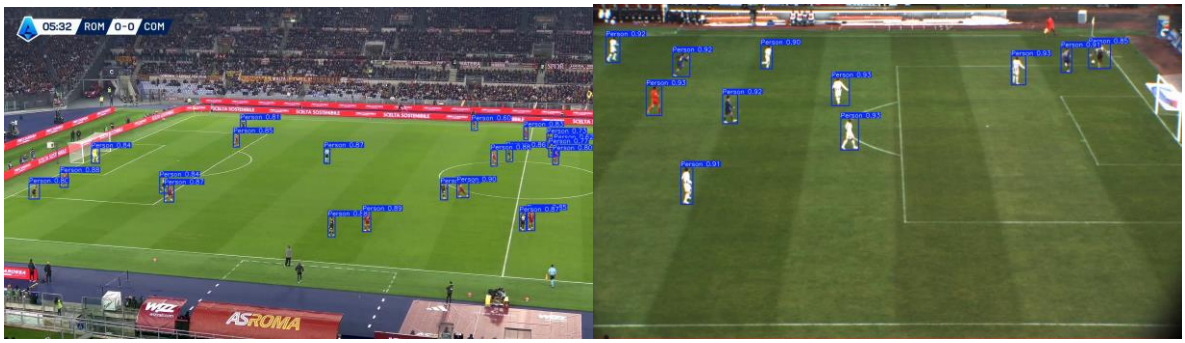


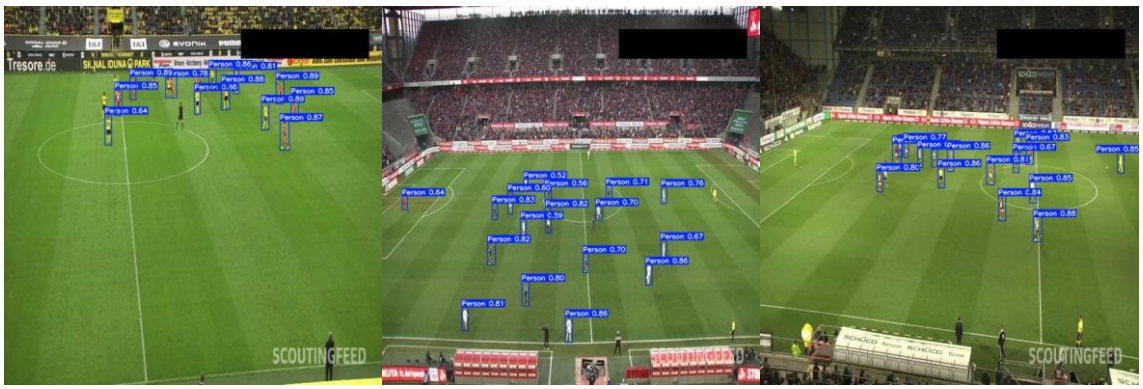


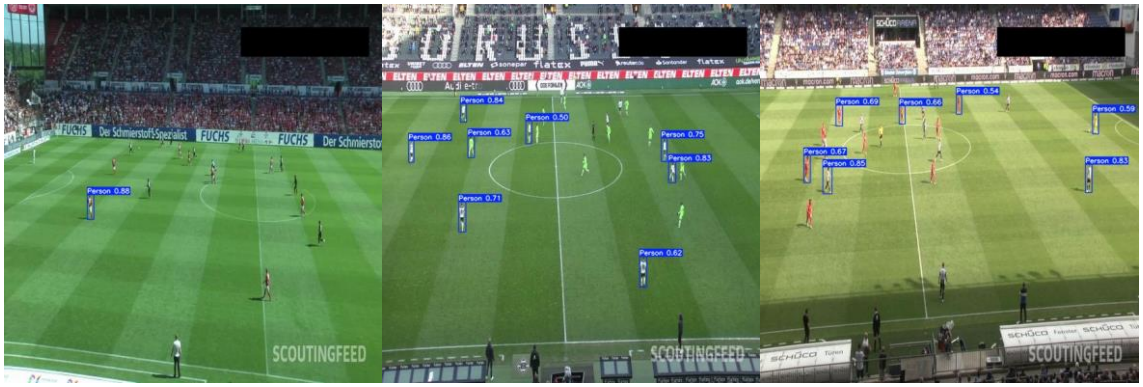


5.1 Qualitative results

To further assess the generalization capability of the proposed detector, we also evaluate it qualitatively on three additional sequences: one from the same match as sequence A, one from the match of sequence B, and one from a completely different game. Although these sequences are not annotated and therefore no quantitative metrics are reported, the visual results confirm that the fine-tuned YOLO model consistently detects players across different viewpoints, broadcast styles, and match contexts. These examples illustrate the robustness of the approach beyond the annotated test set.







The figure illustrates the model's generalization ability across different examples. The first row shows two representative frames, each taken from the same match as the sequence from which A and B were extracted. The second and the third rows report results on a different dataset containing multiple matches. In the second row, the model still performs reasonably well on matches with visual conditions similar to the training sequences. In contrast, the third row shows failure cases from matches with significantly different conditions.

6. Conclusions

In conclusion, our experiments show that classical computer vision pipelines are effective only under favourable viewpoints, while learning-based detectors offer far superior robustness and accuracy. The ROI improves cumulative precision but does not resolve high-confidence false positives in challenging scenes. The fine-tuned YOLO model achieves near-perfect performance across both sequences, demonstrating that modern deep detectors are the most reliable solution for real-world football player detection.

7. Limitations and Future Works

In future work, we plan to further improve generalization by fine-tuning on a broader variety of matches, viewpoints, and broadcast styles, enabling the detector to adapt more effectively to unseen scenarios. Although the fine-tuned model already performs well on sequences from completely different matches, some residual errors remain, as expected when the network encounters visual patterns never observed during training. Extending the training set and exploring transformer-based architectures may further enhance robustness and cross-dataset transferability.